

UNIVERSIDADE FEDERAL DE MINAS GERAIS
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Redes de Computadores (Turma TM)
2º Semestre de 2026
Professor: Luiz Filipe Menezes Vieira (*lfvieira@dcc.ufmg.br*)
Monitor: André Garcia (*andregarcia@dcc.ufmg.br*)

Data de entrega: **21/09/2026**

Trabalho Prático 1: Jogo Termo

O *Termo* é um popular jogo de palavras no qual é preciso adivinhar uma palavra de 5 letras em poucas tentativas. Neste trabalho, você desenvolverá uma versão do jogo baseada na arquitetura **cliente-servidor**: o servidor guarda uma palavra secreta, e o cliente realiza tentativas para adivinhá-la. A cada tentativa, o servidor retorna dicas sobre a precisão das letras, orientando o próximo palpite do jogador.

1 O Projeto

O objetivo deste trabalho é desenvolver um sistema do jogo de palavras (*Termo*) baseado na mecânica de tentativa e erro com *feedback* posicional, inspirada no popular jogo de acerto de palavras. O sistema deve seguir o modelo **cliente-servidor**, permitindo que o usuário tente descobrir a palavra secreta gerada ou armazenada pelo servidor.

1.1 Dinâmica do Jogo

A partida ocorre através de rodadas de interação:

- O **servidor** inicializa o jogo carregando uma palavra secreta (composta por uma sequência de 5 letras, sem acentos).
- O **cliente** envia um palpite (*guess*) com a mesma quantidade de caracteres da palavra original.
- A cada tentativa, o **servidor** processa a mensagem e retorna um *feedback* detalhado:
 - **Acertos Exatos:** Quantidade de letras que estão na posição correta.
 - **Acertos de Caractere:** Quantidade de letras que existem na palavra, mas estão em posições erradas.
- O jogo termina quando o cliente acerta a sequência completa, encerrando a conexão.

1.2 Requisitos Técnicos

Para garantir que o sistema resista às interferências da rede e siga os padrões do Departamento de Ciência da Computação da UFMG, as seguintes restrições devem ser seguidas:

- **Linguagem:** Implementação obrigatória em C, utilizando a interface POSIX de *sockets*. O uso de C++ ou bibliotecas de alto nível é estritamente proibido.
- **Ambiente:** O sistema deve ser desenvolvido e testado exclusivamente para ambiente **Linux**. Bibliotecas específicas de outros sistemas operacionais (como *winsock.h*) não serão aceitas.
- **Rede:** A comunicação deve ser estabelecida via protocolo **TCP**, garantindo a entrega ordenada dos palpites. O sistema deve suportar tanto **IPv4** quanto **IPv6**.
- **Interoperabilidade:** As mensagens devem seguir um protocolo estrito (definido na Seção de Comunicação) para que o cliente de um aluno consiga jogar no servidor de outro.

- **Compilação:** O projeto deve ser compilado exclusivamente através de um **Makefile** funcional, com a simples execução do comando **make** (sem parâmetros adicionais), gerando os binários **client** e **server** na raiz do projeto. Trabalhos que não compilem dessa forma serão zerados (ver Seção de Avaliação).

1.3 Parâmetros de Inicialização

Os executáveis devem aceitar os seguintes argumentos via linha de comando para permitir testes automatizados:

Servidor: `./server <protocolo> <porta> <palavra>`

Onde `<protocolo>` pode ser `v4` ou `v6` e `<palavra>` é a sequência de 5 letras (de A a Z, sem acentos) que deve ser descoberta.

Cliente: `./client <endereço_ip> <porta>`

Onde `<endereço_ip>` é o endereço do servidor.

2 O Jogo

O servidor aguarda que o cliente envie um palpite para tentar descobrir a palavra. O feedback é individual para cada uma das 5 posições, conforme as categorias da Tabela 1:

Table 1: Categorias de Feedback por Posição

Tipo	Status do Caractere	Significado
2	<i>Posição Correta</i>	A letra daquela posição está correta.
1	<i>Caractere Presente</i>	A letra existe na palavra, mas em outra posição.
0	<i>Caractere Ausente</i>	A letra não faz parte da palavra.

O resultado da tentativa é mapeado para os seguintes estados do sistema:

- **1 (Vitória):** O cliente enviou as 5 letras corretas nas posições exatas. O jogo é encerrado com sucesso.
- **0 (Palpite Incorreto):** O palpite de 5 letras está incorreto. O servidor retorna o vetor de dicas (Tabela 1) para orientar o próximo palpite.
- **-1 (Erro de Protocolo):** O cliente enviou um palpite com **tamanho diferente de 5 caracteres** ou utilizou caracteres fora do conjunto permitido (letras de A a Z, sem acentos).

3 Tratamento de Erros e Robustez

A estabilidade do sistema é garantida pelo tratamento rigoroso de entradas inválidas. Caso o cliente envie um palpite com tamanho diferente de 5 letras ou que contenha símbolos não alfabéticos, o servidor não deve encerrar a conexão abruptamente.

Em vez disso, o servidor deve descartar o pacote inválido, retornar o código de erro correspondente (Tipo -1) e aguardar um novo envio. Enquanto isso, o cliente deve tratar esse retorno e exibir na tela para o usuário a seguinte mensagem de alerta:

“**ERRO: Insira uma sequência de 5 caracteres de A a Z!**”

Dessa forma, tentativas inválidas não devem resetar o progresso da rodada atual, permitindo que o usuário continue a partida sem penalidades de conexão.

Palpites contendo letras minúsculas (*a-z*) não devem ser tratados como erro de protocolo. O sistema deve convertê-las para maiúsculas antes da validação, prosseguindo o jogo normalmente.

4 Protocolo de Comunicação

4.1 Estrutura da Mensagem

A comunicação via *sockets* entre o cliente e o servidor será realizada através de uma estrutura de mensagem padronizada. Essa estrutura garante que os palpites alfabéticos e os *feedbacks* posicionais sejam interpretados corretamente, seguindo as normas de interoperabilidade do projeto.

Listing 1: Definição do Protocolo Termo

```
#define MSG_SIZE 128

typedef enum {
    MSG_START,      // Servidor solicita a palavra de acesso
    MSG_GUESS,      // Cliente envia o palpite de 5 letras
    MSG_FEEDBACK,   // Servidor retorna as dicas (0, 1 ou 2)
    MSG_WIN,        // Servidor informa que a palavra foi descoberta
    MSG_ERROR,      // Servidor reporta erro de formato/tamanho
    MSG_EXIT        // Encerramento da conexao
} MessageType;

typedef struct {
    int type;        // Tipo da mensagem (MessageType)
    int guess[5];    // Vetor com as 5 letras enviadas pelo cliente
    int feedback[5]; // Dicas retornadas pelo servidor (2, 1 ou 0)
    int attempts;    // Contador de tentativas realizadas
    int win_status;  // 1 para vitória, 0 para em jogo, -1 para erro
    char message[MSG_SIZE]; // Mensagem de texto para logs
} GameMessage;
```

Observação: É obrigatório que a mensagem siga exatamente essa estrutura para garantir a interoperabilidade e padronização da comunicação.

4.2 Fluxo de Mensagens

O ciclo de vida do jogo segue a ordem rigorosa de requisição e resposta:

1. **Servidor → Cliente:** Envia `tipo = MSG_START` para inicializar as tentativas.
2. **Cliente → Servidor:** Envia `tipo = MSG_GUESS` com o vetor `guess[5]` preenchido com letras de A a Z.
3. **Servidor → Cliente:** Responde com `tipo = MSG_FEEDBACK`, preenchendo o vetor `feedback[5]` com as dicas e atualizando o contador de `attempts`.
4. **Servidor → Cliente:** Caso as 5 letras estejam na posição correta, envia `tipo = MSG_WIN` e encerra o jogo.

5 Execução

5.1 Execução do Servidor

Para inicializar o servidor, é necessário especificar o protocolo de rede, a porta de escuta e a palavra secreta. Por exemplo, para utilizar IPv4 na porta 51511, com a palavra TERMO, utilize o comando:

```
./server v4 51511 TERMO
```

5.2 Execução do Cliente

Para conectar o cliente a um servidor ativo, deve-se informar o endereço IP e a porta de destino. Por exemplo, para conectar-se ao servidor local (*localhost*) na porta 51511:

```
./client 127.0.0.1 51511
```

6 Exemplos de Uso

Para facilitar a implementação e garantir a interoperabilidade entre os sistemas, apresentamos um cenário de partida via IPv4. Neste exemplo, o servidor é iniciado com a palavra **TERMO**. O jogo termina quando a palavra é descoberta.

Terminal do Cliente	Terminal do Servidor
	\$./server v4 51511 TERMO Servidor iniciado em modo IPv4 na porta 51511.
\$./client 127.0.0.1 51511	Cliente Conectado
Insira seu palpite: > NORTE Dica: _ * R * * Tentativas realizadas: 1	
Insira seu palpite: > TERMOS ERRO: Insira uma sequência de 5 caracteres de A a Z!	
Insira seu palpite: > MORER Dica: * * R * _ Tentativas realizadas: 2	
Insira seu palpite: > TERMO Parabéns! Você venceu!	Cliente Desconectado

7 Padronização da Saída (Interface)

Para possibilitar a correção automática e a interoperabilidade entre diferentes implementações, os programas devem seguir rigorosamente o padrão de exibição de texto descrito nesta seção. **Qualquer caractere extra, espaços adicionais ou mensagens personalizadas fora do padrão podem resultar em falha nos testes automatizados.**

7.1 Saída do Cliente

O cliente deve interagir com o usuário e exibir o feedback do servidor seguindo estas regras: **Exibição da Dica:** O feedback posicional deve ser impresso em uma única linha, precedido por “Dica: ” e, em sequência, os símbolos da dica separados por espaço. Para cada uma das 5 posições, utilize:

- <letra> para Posição Correta.
- * para Caractere Presente (mas em posição errada).
- _ (underline) para Caractere Ausente **ou esgotado**.

Regra de contagem: Cada letra da palavra só pode ser “usada” uma vez no feedback. Se o palpite contém a letra X mais vezes do que ela aparece na palavra, as ocorrências excedentes de X no palpite devem receber _, mesmo que X exista na palavra, pois todas as suas ocorrências já foram atribuídas a outras posições do palpite.

Table 2: Exemplos de Feedback para Casos Específicos

Palavra Secreta	Palpite	Dica (Saída)	Explicação Lógica
MORTA	MORTE	M O R T _	A letra ‘E’ do palpite não faz parte da palavra, recebendo (-).
CASAS	SACAS	* A * A S	Os ‘A’s das posições 2 e 4 e o ‘S’ da pos. 5 são exatos; o ‘S’ (pos. 1) e o ‘C’ (pos. 3) existem na palavra, mas em outras posições (*).
OSSOS	SOCOS	* O _ * S	O ‘O’ da pos. 2 é exato; o segundo ‘O’ está presente; o ‘S’ da pos. 5 é exato; o ‘C’ é ausente e o ‘S’ da pos. 1 encontra o pool já consumido pelas posições exatas (-).
ARARA	RATAA	* * _ * A	A palavra tem três ‘A’s: um exato (pos. 5) e dois presentes (pos. 2 e 4); o ‘T’ é ausente e o ‘R’ da pos. 1 está presente (*).
PIANO	FESTA	_ _ _ _ *	A letra ‘A’ do palpite existe na palavra.
SONSO	SSSSS	S _ _ S _	A palavra tem dois ‘S’ nas posições exatas; as demais ocorrências de ‘S’ são esgotadas (-).

Contador de Tentativas: Após a dica, o cliente deve imprimir o número total de palpites enviados da seguinte forma:

Tentativas realizadas: N

Mensagens de Estado:

- Erro de formato: `ERR0: Insira uma sequência de 5 caracteres de A a Z!`
- Vitória: `Parabéns! Você venceu!`

7.2 Saída do Servidor

Embora o servidor opere em segundo plano, ele deve registrar eventos básicos para depuração:

- Ao iniciar: `Servidor iniciado em modo <IPv4/IPv6> na porta <porta>`
- Ao receber conexão: `Cliente conectado`
- Ao encerrar: `Cliente desconectado`

8 Informações sobre a Entrega

A entrega deve ser feita no formato ZIP, com o nome seguindo o seguinte padrão:

TP1_NOME_MATRICULA.zip

8.1 Penalidade por Atraso

O TP deverá ser entregue de acordo com a data especificada no Moodle. Entregas atrasadas serão aceitas, porém sujeitas a uma penalidade percentual sobre a nota final, calculada pela fórmula:

$$Penalidade(\%) = 2^{(d-1)}/0.16$$

onde d é o número de dias corridos de atraso em relação ao prazo de entrega (contando o primeiro dia após o prazo como $d = 1$). A penalidade calculada é subtraída diretamente da nota final do trabalho, podendo zerá-lo caso ultrapasse 100%.

8.2 Documentação

Cada aluno deve entregar uma documentação em PDF de até **4 páginas**. A documentação deve discutir os desafios, dificuldades e imprevistos do projeto, bem como as soluções adotadas para os problemas. É imprescindível que vocês discutam todos os desafios que encontrarem, isso será muito importante para a avaliação.

8.3 Código

Além da documentação, cada aluno deve entregar o **código fonte em C** e um **Makefile** para a compilação do programa. Seu servidor será corrigido de forma semi-automática por uma bateria de testes. Para garantir o pleno funcionamento da correção, o código de vocês deve seguir as seguintes instruções:

- O Makefile deve compilar o cliente em um binário chamado “client” e o servidor em um binário chamado “server” dentro da raiz do projeto.
- Seu programa deve ser compilado com a execução do comando make, ou seja, sem a necessidade de parâmetros adicionais. Para isso, você precisa escrever um Makefile funcional

Procure escrever seu código de maneira clara, com comentários pontuais e bem identados. Isto facilita a correção do monitor e tem impacto positivo na avaliação.

Exemplo de Makefile

O exemplo abaixo é apenas uma referência de estrutura básica — adapte-o à organização de arquivos do seu projeto (nomes de fontes, headers, subpastas, etc.):

Listing 2: Exemplo de Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -g

all: client server

client: client.c
    $(CC) $(CFLAGS) -o client client.c

server: server.c
    $(CC) $(CFLAGS) -o server server.c

clean:
    rm -f client server *.o

.PHONY: all clean
```

9 Avaliação

O sistema será avaliado de forma automatizada, considerando o suporte aos protocolos IPv4 e IPv6. A distribuição da pontuação seguirá os critérios detalhados na Tabela 3.

Será adotada a média harmônica entre as notas da documentação e da execução, o que implica que a nota final será **zero** se uma das partes não for apresentada.

Atenção: Se não for possível compilar seu trabalho usando o comando make, sua nota final será automaticamente zero! Não serão aceitas entregas fora do prazo!

Table 3: Distribuição de pontos por critério de avaliação.

Critério	Peso
Inicialização correta do servidor	10%
Conexão do cliente estabelecida com o servidor	15%
Funcionamento das jogadas (Cliente → Servidor)	10%
Funcionamento correto das dicas	10%
Contabilização de tentativas	10%
Resultado do encerramento do jogo	5%
Fechamento adequado de conexões	10%
Tratamento de erros e exceções	5%
Formatação e impressão de mensagens	5%
Documentação técnica	20%
Total	100%

10 Integridade Acadêmica

Este projeto deve ser produzido **por você!** É permitido e incentivado a discussão de conceitos com colegas, testar sua implementação contra a deles e tirar dúvidas no fórum, mas o código deve ser 100% autoral.

- **Plágio entre alunos** (cópia de código): nota zero para todos os envolvidos.
- **Uso de código gerado por LLMs** (ChatGPT, Copilot, etc.): nota zero. Ferramentas de análise estrutural de código serão utilizadas na correção.

11 Materiais de estudo

1. Capítulo 2 e 3 do livro sobre programação com sockets disponibilizado no Moodle.
2. Vídeo aula: “Programação com sockets, professor Ítalo Cunha”